

Requirement Formalization in Modelica

Lena Buffoni, Linköping University
(with material from Wladimir Schamai)



WHAT IS A REQUIREMENT?



Definitions

“Software requirements express the *needs* and *constraints* placed on a software product that contribute to the solution of some real-world problems.”

(Kotonya and Sommerville, 2000)

“A requirement is something the product *must do* or a *quality it must have*.”

(Suzanne & James Robertson, 2006)

Examples

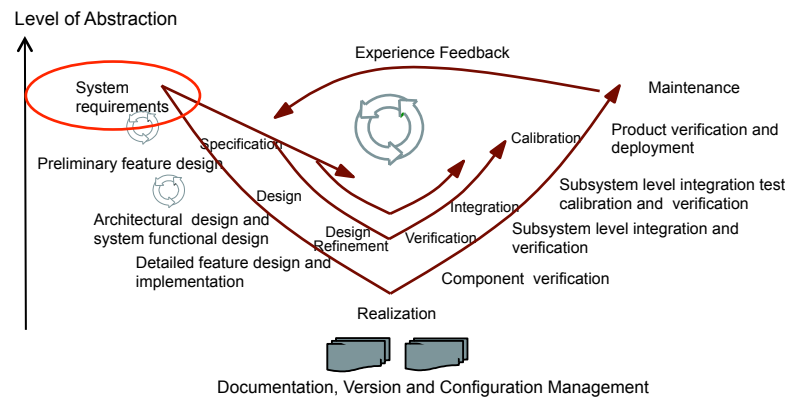
- “*The torque of any ADGB electrical motor shall not be superior to 20 N.m for more than 1 sec.*”
- “*At least 2 pumps shall be in operation at any time*”
- “*In the absence of any BPS component failure or in the presence of a single sensor failure, when the BPS is not under maintenance, and in case of MPS loss, at least two of Set2 to Set5 must be powered within 40 s*”

Some Requirements Engineering Literature

- Kotonya G. and Sommerville, I. Requirements Engineering: Processes and Techniques. Chichester, UK: John Wiley & Sons
- [Software Requirements Engineering Methodology \(Development\)](#) Alfor, M. W. and Lawson, J. T. TRW Defense and Space Systems Group. 1979.
- Thayer, R.H., and M. Dorfman (eds.), System and Software Requirements Engineering, IEEE Computer Society Press, Los Alamitos, CA, 1990.
- Royce, W.W. 'Managing the Development of Large Software Systems: Concepts and Techniques', IEEE Westcon, Los Angeles, CA> pp 1-9, 1970. Reprinted in *ICSE '87, Proceedings of the 9th international conference on Software Engineering*.
- [Requirements bibliography](#) Reviewed November 10th 2011
- Sommerville, I. *Software Engineering*, 7th ed. Harlow, UK: Addison Wesley, 2006.
- Ralph, Paul (2012). "[The Illusion of Requirements in Software Development](#)". *Requirements Engineering*.

WHY DO WE WANT TO MODEL REQUIREMENTS?

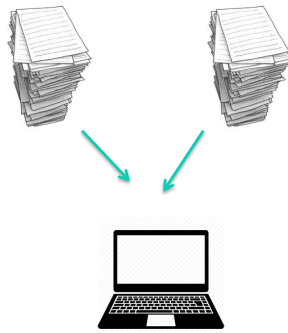
Requirement formalization in product life-cycle



Why are Requirements Important?

- Requirements Engineering is an important part of the early phases in system design V
- Errors in system requirements cause large cost increase in product development, or even completely failed projects
- Requirements engineering should be an integrated part of the development tool chain
- Some resources:
 - International Requirements engineering conference home page: <http://requirements-engineering.org/>
 - Example of requirements engineering journal: **Requirements Engineering**, Springer Verlag, <http://link.springer.com/journal/766>

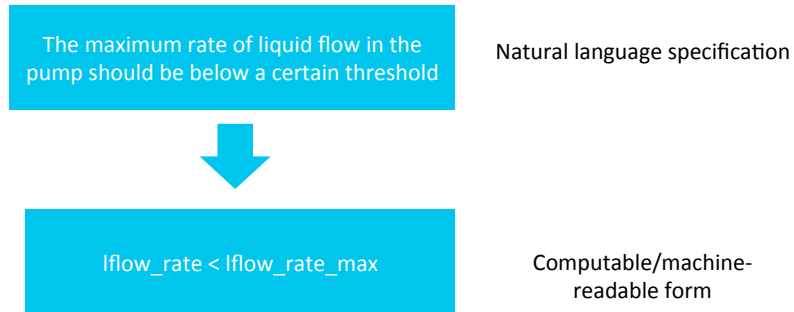
What is requirement analysis used for?



- Detect and resolve conflicts between requirements and different levels of requirement specification
- Define the bounds of normal system behaviour
- Define interaction with the environment
- Break down high-level requirements into more detailed requirements

HOW DO WE REPRESENT REQUIREMENTS?

Can all requirements be formalized?



Can all requirements be formalized?

Functional: specify a behavior or function

- *“The pump shall not cavitate”*

Non-functional: system constraints performance constraints

- *“The controller code should be in C”*
- *“The response time should be less than 0.1s”*

13

Why model requirements in Modelica?

- We can have them alongside the component Model
- Declarative style of Modelica well suited to requirement expression
- Integrated in the tool chain
- Can be verified during simulation

14

What do need to represent requirements?

- We need to identify requirements in Modelica
- Requirements must have a **status**
- Requirements do **not modify** the physical model
- For specific uses, we need to know which models are requirements, e.g.
 - automatic model composition,
 - requirement verification,
 - ...
- Requirement models can (possibly) contain extensions for expressing requirements in a more requirement-designer friendly way (FORM-L macros for example)

Status of a requirement

15

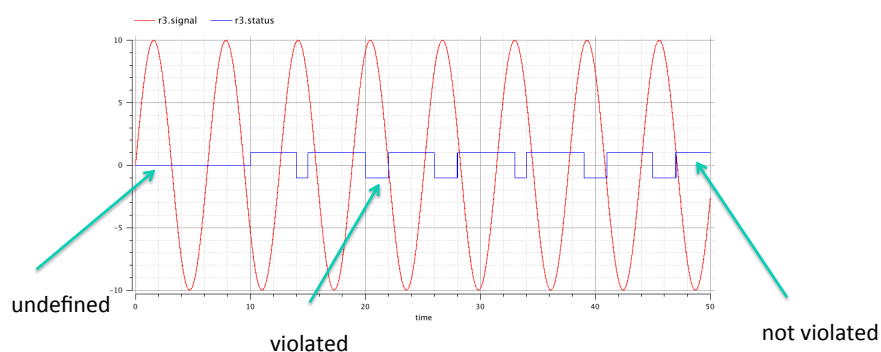
Example requirement: **when the pump is on the flow must be above a minimum threshold**

The **status** variable is 3-valued and applies to a specific instant in time:

- **Violated** : the requirement is applicable and the conditions of the requirement are not fulfilled – the pump is on and the flow is below the minimum required
- **Satisfied** : the requirement is applicable and the conditions of the requirement are fulfilled – the pump is on and the flow is above the minimum required
- **Not_applicable** : the requirement cannot be applied or is not relevant - the pumps are off

Status over simulation time

16



Status changes over time, so we need to track it over the course of the simulation

Example 1 - LimitInFlow

17

```

model LimitInFlow "A2: If pump is on then in-flow is
                        less than maxLevel"

    extends Requirement;
    //qOut from the Source component
    input Real liquidFlow;
    input Boolean pumpOn;
    parameter Real maxLevel = 0.05;
equation
    if (pumpOn) then
        if (liquidFlow < maxLevel) then
            status = Status.not_violated;
        else
            status = Status.violated;
        end if;
    else status = Status.not_applicable; end if;
end LimitInFlow;

```

Example (2) : No pump shall cavitate

18

```

package Requirements
model Req
    /* number of cavitating pumps*/
    input Real numberOfCavPumps = 0;
    /* min. number of operating pumps */
    constant Integer maxNumOfCavPumps = 0;
    /*indication of requirement violation, 0 = means not evaluated */
    output Status status(start=status_not_applicable,fixed=true);

    algorithm
        if numberOfCavPumps > maxNumOfCavPumps then
            status := Status.violated;
        else
            status := Status.not_violated;
        end if;
    end Req;
end Requirements;

```

☑ Standard Modelica

19

FORM-L

- FORM-L a language for property expression proposed by EDF
- Aims to make requirement expression easier for engineers
- Example :
 Form-L: during (On and (MPSVoltage > V170))
 check no (Off becomes true);
- Integrate it with Modelica to breach semantic gap between how engineers and modelers express requirements

20

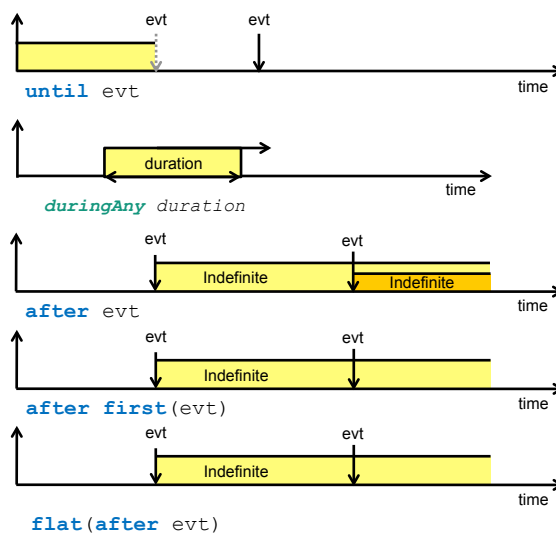
Property modeling: structure of a property

- Properties are divided into 4 main parts:
 WHERE / WHEN / WHAT / HOW_WELL
- **WHERE**: spatial locator. Builds the sets of objects defined by static properties.
- **WHEN**: temporal locator. Locate properties in time on sets built with spatial locators.
- **WHAT**: condition to be verified at the specified **WHERE** and during the specified **WHEN**.
- **HOW_WELL**: probabilistic condition of the condition to be verified

FORM-L and Modelica

- Mapping Form-L constructs to Modelica whenever possible
- Defining some dedicated types (properties, requirements, conditions...)
- Defining a library of Time-Locators in Modelica

Time Locators



Special blocks that define when a requirement should be verified (after, until, afterFor...)

Many more specified in the FORM-L documentation by N. Thuy

Calling blocks as functions

- proposed standardized syntax:
`x = blockname().y`
- The compiler will automatically generate and instantiate the corresponding blocks
- Supported by OpenModelica and Dymola

Time Locator Libraries

- Requirement – small library developed by me ☺ for testing requirements with OpenModelica
- Property2 library – developed by Martin Otter at DLR

25

Modeling with Requirement library:

```
// Form-L: during (On and (MPSVoltage > V170)) check no (Off
becomes true);
model Req1b_is_on extends Requirement;
input Real MPSVoltage;
input Boolean on;
input Boolean off;
equation
    status = if (on and (MPSVoltage > V170)) then
        if(off) Status.violated else Status.not_violated
    else Status.not_applicable;
end Req1bis;
```

26

Textual requirement modeling – Using TimeLocator WithinAfter

```
model R3 "requirement R3 is during any s6 check avg(signal) < 20;"
    extends Requirement;
    input Real signal;
    equation
        if time < 6 then
            status = undefined;
        else
            if DuringAnyAverage(timeWindow = 6, value = signal, thr = 2).out
            then
                status = not_violated;
            else
                status = violated;
            end if;
        end if;
    end R3;
```

DuringAnyAverage time locator

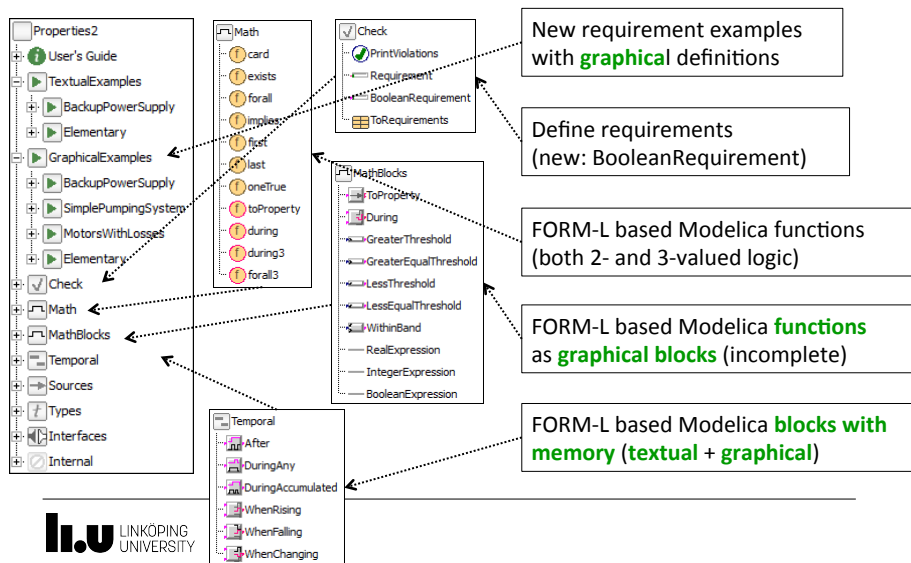
```

block DuringAnyAverage "Returns true if the average of the values in a
sliding window of sizeWindow is below the threshold"
  extends TimeLocator;
  parameter Real timeWindow = 10;
  parameter Real sampleFrequency = 1;
  parameter Real thr;
  input Real value;
  protected
    Real buffer[size];
    Integer counter;
    parameter Integer size = integer(floor(timeWindow /
sampleFrequency)) + 1;
  algorithm
    when sample(0, sampleFrequency) then
      buffer[counter + 1] := value "store values in buffer";
      counter := mod(counter + 1, size-1);
      if time > timeWindow then
        out := average(buffer) <= thr;
      else
        out := true;
      end if;
    end when;
  end DuringAnyAverage;

```

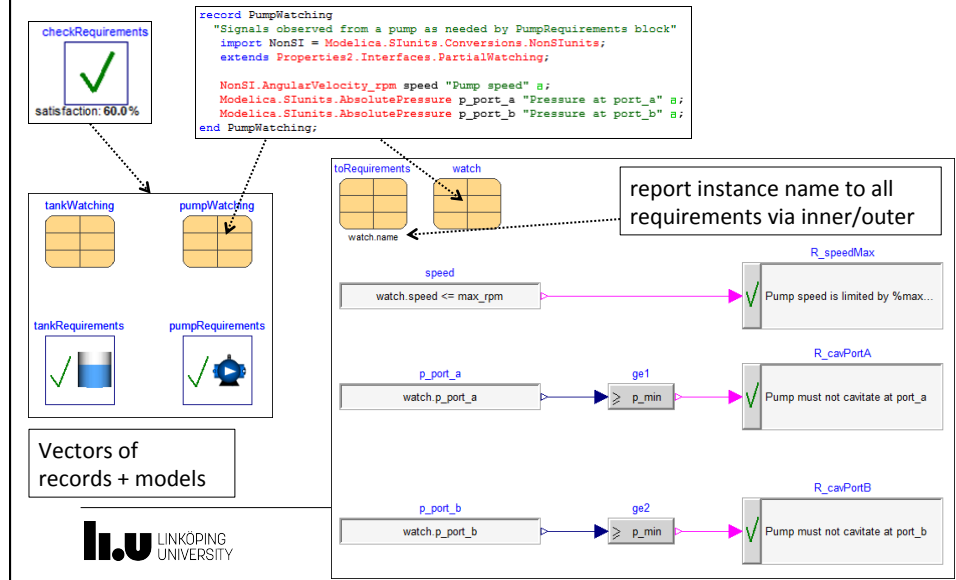
Improved Properties2 library - Details

28



Example SimplePumpingSystem: Definition of graphical requirements

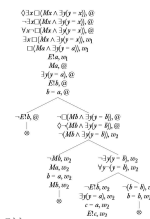
29



HOW DO WE USE REQUIREMENTS FOR VERIFICATION?

Formal model checking methods

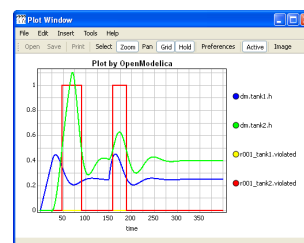
- Formal language specification (ESTERL, petri nets, temporal languages)
- Verification tools (Coq, certified code generators...)
 - + Prove correctness theoretically
 - + Give theoretical guarantees
 - Heavy/expensive to use
 - What can be verified is limited



Tautology:	$P \rightarrow (Q \rightarrow (P \wedge Q))$
Necessitation:	$\Box(P \rightarrow (Q \rightarrow (P \wedge Q)))$
Modal axiom:	$\Box(P \rightarrow (Q \rightarrow (P \wedge Q))) \rightarrow (\Box P \rightarrow \Box(Q \rightarrow (P \wedge Q)))$
Modus ponens:	$\Box P \rightarrow \Box(Q \rightarrow (P \wedge Q))$
Modal axiom:	$\Box(Q \rightarrow (P \wedge Q)) \rightarrow (\Box Q \rightarrow \Box(P \wedge Q))$
Classical arg:	$\Box P \rightarrow (\Box Q \rightarrow \Box(P \wedge Q))$
Classical arg:	$(\Box P \wedge \Box Q) \rightarrow \Box(P \wedge Q)$

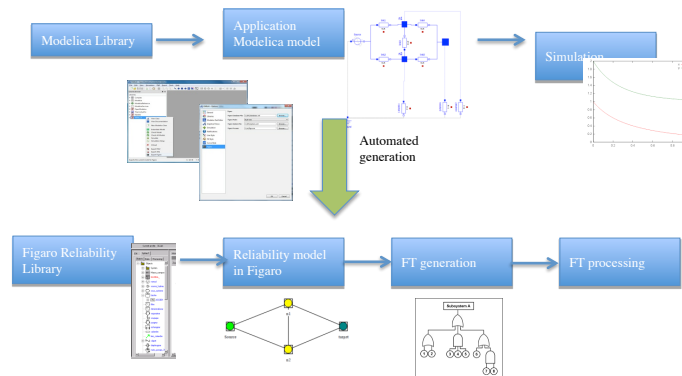
Simulation based model checking methods

- Simulate a certain number of scenarios and verify requirements dynamically
- More flexible
- Find the presence not the absence of bugs
- Can still give good guarantees

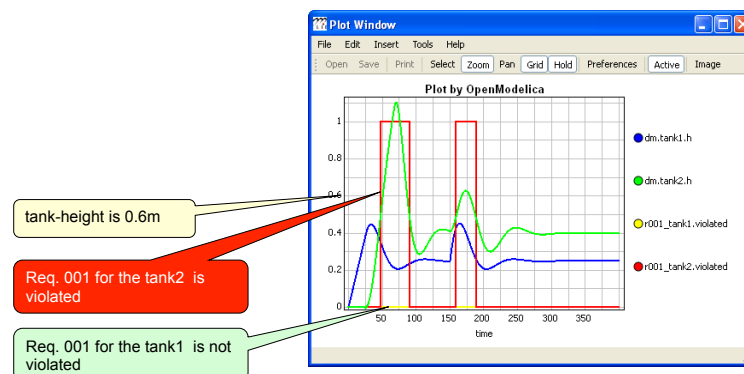


Failure Mode and Effects Analysis (FMEA) in OM

- Modelica models augmented with reliability properties can be used to generate reliability models in Figaro, which in turn can be used for static reliability analysis
- Prototype in OpenModelica integrated with Figaro tool (which is becoming open-source)



Dynamic Requirements Evaluation



Verification example

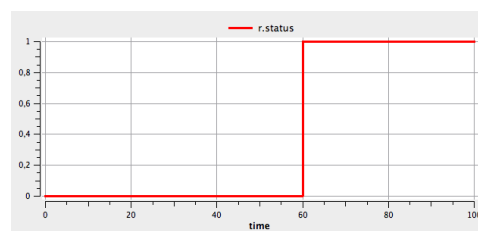
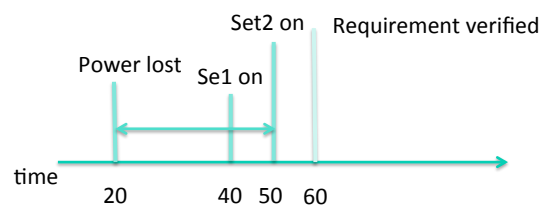
```

block R2 "after the power is lost, at least 2 powerSets
must be on within 40s"
  extends Requirement;
  input Boolean[5] isOn;
  input Boolean powerLoss;
  output Integer status(start = 0);
  Boolean wA;
  equation
    wA = withinAfter(40, powerLoss);
  when wA then
    status = if countTrue(isOn) >= 2 then
not_violated else violated;
  elsewhen not wA then
    status = undefined;
  end when;
end R2;

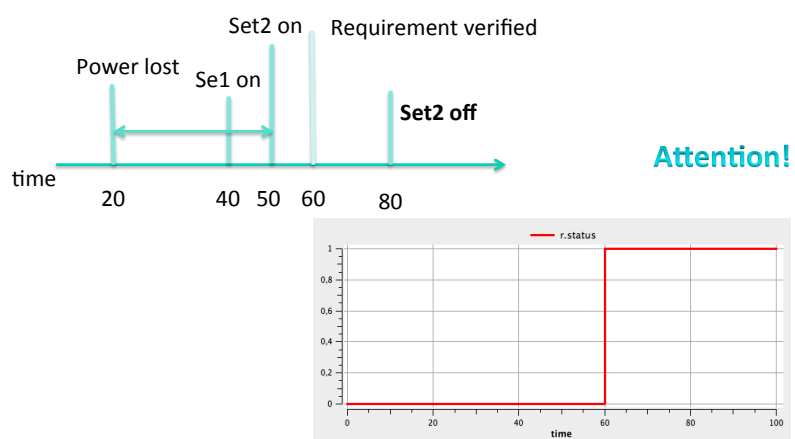
```



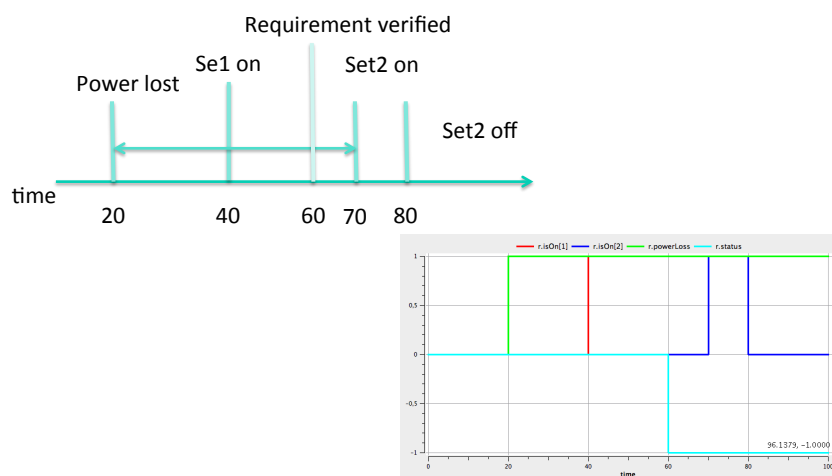
Scenario 1



Scenario 1



Scenario 2



Example MPS: Log after simulation

name of requirement

Short summary (% from DS library)

instance name of model
(where requirement violated)

requirement in textual form

time of first violation

```
----- 31.3 % of MainPowerSupplyRequirements requirements are satisfied -----
Requirements violated (5 of 8):
  R1 in MPS at 10 s: MPS CAN be declared Off when the voltage gets below 170 V
  R2 in MPS at 25 s: MPS MUST be declared Off when the voltage gets below 160 V
  R3 in MPS at 15 s: MPS CAN be declared On again when the voltage gets above 200 V
  R4 in MPS at 10 s: MPS MUST be declared On again when the voltage gets above 210 V
  R7 in MPS at 0 s: MPS CAN be declared Available again when On for more than 8 consecutive seconds

Requirements untested (1 of 8):
  R8 in MPS: MPS MUST be declared Available again when On for more than 10 consecutive seconds

Requirements satisfied (2 of 8):
  R5 in MPS: MPS CAN be declared Unavailable when it has been Off for more than 4 consecutive seconds
  R6 in MPS: MPS MUST be declared Unavailable when it has been Off for more than 6 consecutive seconds
```

Output configurable:

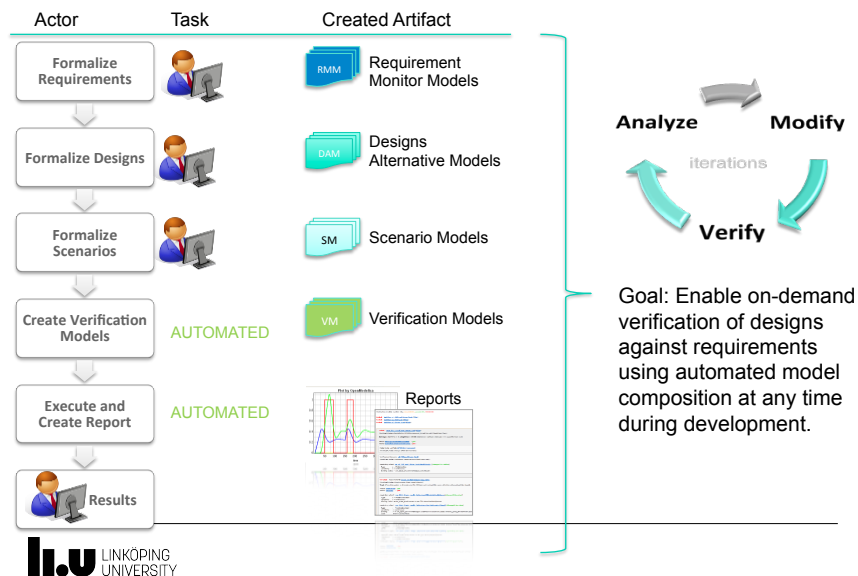
Violated, untested and/or satisfied requirements

3

Dynamic Requirement Verification vs System Design

- **Formalized Requirements** that should be verified
- **System Model, i.e., Design Alternative Model**, for which the requirements should be verified
- **Application scenarios** with the system model for which the requirements should be verified

vVDR Method – virtual Verification of Designs vs Requirements



Example: System and Requirement

- A system contains several pumps,
 - Requirement: *"No pump within the system shall cavitate at any time"*
- System model:

```

model SRI
  Machines.PumpA PO1;
  Machines.PumpB PO2;
  Machines.PumpA PO3;
end SRI;

package Machines
  model PumpA
    // Volumetric mass flow-rate inside the pump
    Real Qv = 1;
    // Pressure at the inlet (C1 is the fluid connector at the inlet)
    Real C1_P = 1;
    // Pressure at the outlet (C2 is the fluid connector at the outlet)
    Real C2_P = 1;
  end PumpA;

  model PumpB
    // Volumetric mass flow-rate inside the pump
    Real Qv = 1;
    // Pressure at the inlet (C1 is the fluid connector at the inlet)
    Real C1_P = 1;
    // Pressure at the outlet (C2 is the fluid connector at the outlet)
    Real C2_P = 1;
    // Minimum pressure inside the pump
    Real Pmin = 20;
  end PumpB;
end Machines;

```

Formalized requirement:

```

package Requirements
  model Req
    /* number of cavitating pumps */
    input Real numberOfCavPumps = 0;
    /* min. number of operating pumps */
    constant Integer maxNumOfCavPumps = 0;
    /* indication of requirement violation, 0 = means not evaluated */
    output Integer status(start=0, fixed=true);
  end Req;

  algorithm
    if numberOfCavPumps > maxNumOfCavPumps then
      status := 2 "2 means violated";
    else
      status := 1 "1 means NOT violated";
    end if;
  end Req;
end Requirements;

```

Example: Analysis model

model AnalysisModel

...
Requirements.Req req1;

SRI sri;

end AnalysisModel;

Formalized requirement:

```
package Requirements
model Req
/* number of cavitating pumps */
input Real numberOfCavPumps = 0;
/* min. number of operating pumps */
constant Integer minNumOfCavPumps = 0;
/* indication of requirement violation, 0 = means not evaluated */
output Integer status(start=0, fixed=true);

algorithm
if numberOfCavPumps > minNumOfCavPumps then
status := 2 "2 means violated";
else
status := 1 "1 means NOT violated";
end if;
end Req;
end Requirements;
```

System model:

```
model SRI
Machines.PumpA PO1;
Machines.PumpB PO2;
Machines.PumpA PO3;
end SRI;

package Machines
model PumpA
/* Volumetric mass flow-rate inside the pump
Real Qv = 1;
/* Pressure at the inlet (C1 is the fluid connector at the inlet)
Real C1_P = 1;
/* Pressure at the outlet (C2 is the fluid connector at the outlet)
Real C2_P = 1;
end PumpA;

model PumpB
/* Volumetric mass flow-rate inside the pump
Real Qv = 1;
/* Pressure at the inlet (C1 is the fluid connector at the inlet)
Real C1_P = 1;
/* Pressure at the outlet (C2 is the fluid connector at the outlet)
Real C2_P = 1;
/* Minimum pressure inside the pump
Real Pmin = 20;
end PumpB;
end Machines;
```

Automatic model composition

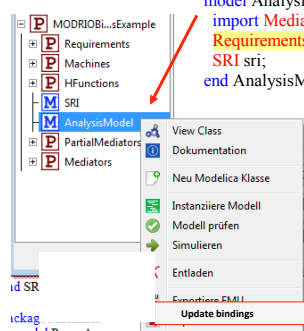
Initial AnalysisModel:

```
model AnalysisModel
import Mediators.*;
Requirements.Req req1;
SRI sri;
end AnalysisModel;
```

Binding:

client instance reference = binding expression

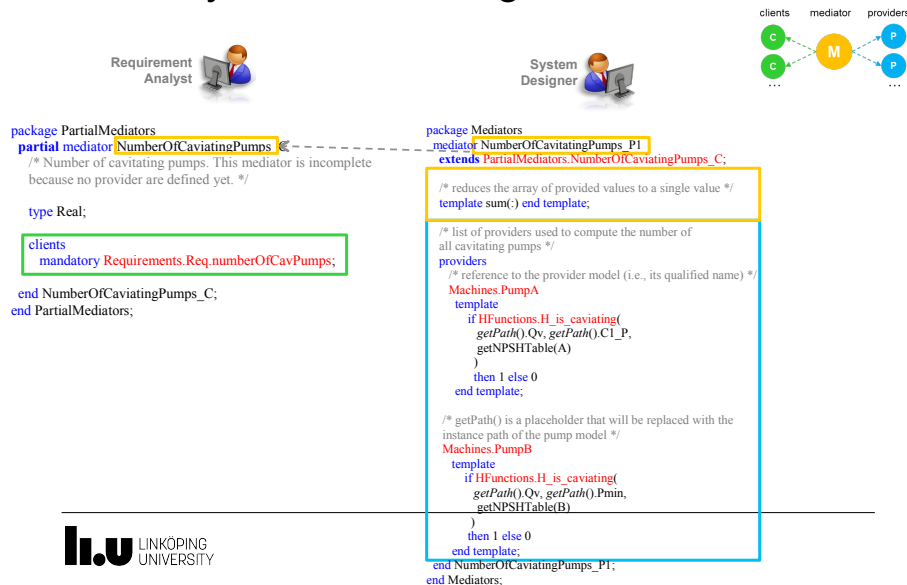
A **binding** is a causal relation which specifies that, at any simulated time, the value given to the referenced client instance shall be the same as the value computed by the right-hand expression.



Updated AnalysisModel model with a binding:

```
model AnalysisModel
import Mediators.*;
Requirements.Req req1(numberOfCavPumps=
sum({
  (if (HFunctions.H_is_cavitating(sri.PO1.Qv, sri.PO1.C1_P, 1))
  then 1 else 0),
  (if (HFunctions.H_is_cavitating(sri.PO2.Qv, sri.PO2.Pmin, 1))
  then 1 else 0),
  (if (HFunctions.H_is_cavitating(sri.PO3.Qv, sri.PO3.C1_P, 1))
  then 1 else 0)}));
SRI sri;
end AnalysisModel;
```

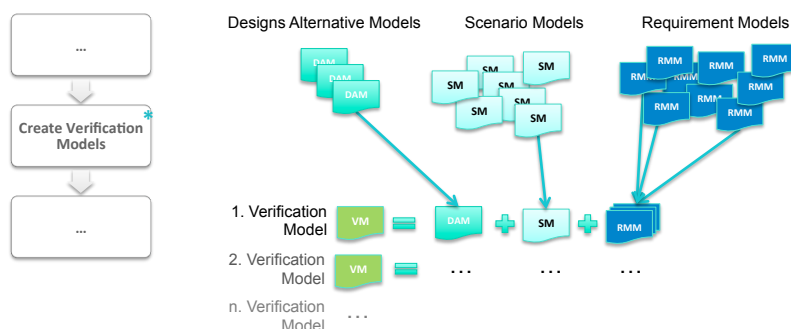
How do we capture the information necessary for automatic generation?



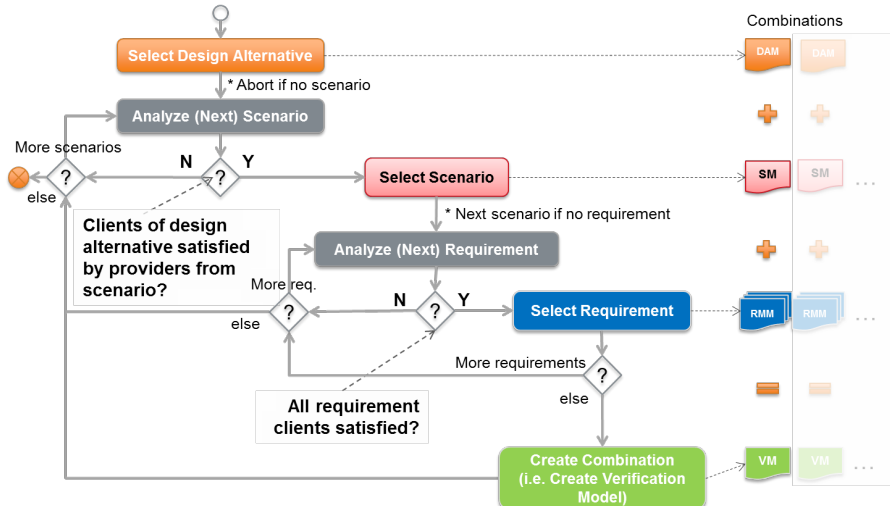
Challenge

We want to verify **different design alternatives** against **sets of requirements** using **different scenarios**. Questions:

- 1) How to **find valid combinations** of **design alternatives**, **scenarios** and **requirements** in order to enable an automated composition of verification models?
- 2) Having found a valid combination: How to **bind all components correctly**?



Generating/Composing Verification Models



Composing Verification Models

- Collect all **scenarios**, **requirements**, import **mediators**
- Generate/compose *verification models* automatically:
 - Select the **system model** to be verified
 - Find all **scenarios** that can stimulate the selected system model (i.e., for each mandatory client check whether the binding expression can be inferred)
 - Find **requirements** that are implemented in the selected system model (i.e., **check** whether for **each requirement** for all mandatory clients binding expressions can be inferred)
- Present the list of scenarios and requirements to the user
 - The user can select only a subset of scenarios or requirements he/she wishes to consider

That's all! Thank you! Questions?

www.liu.se